Google

Co-RedTeam: Orchestrated Security Discovery and Exploitation with LLM Agents

Pengfei He[1 3*], **Ash Fox**[2], **Lesly Miculicich**[1], **Stefan Friedli**[2], **Daniel Fabian**[2], **Burak Gokturk**[1], **Jiliang Tang**[3], **Chen-Yu Lee**[1], **Tomas Pfister**[1] **and Long T. Le**[1]
[1]Google Cloud AI Research, [2]Google, [3] Michigan State University

Large language models (LLMs) have shown promise in assisting cybersecurity tasks, yet existing approaches struggle with automatic vulnerability discovery and exploitation due to limited interaction, weak execution grounding, and a lack of experience reuse. We propose Co-RedTeam , a security-aware multi-agent framework designed to mirror real-world red-teaming workflows by integrating security-domain knowledge, code-aware analysis, execution-grounded iterative reasoning, and long-term memory. Co-RedTeam decomposes vulnerability analysis into coordinated discovery and exploitation stages, enabling agents to plan, execute, validate, and refine actions based on real execution feedback while learning from prior trajectories. Extensive evaluations on challenging security benchmarks demonstrate that Co-RedTeam consistently outperforms strong baselines across diverse backbone models, achieving over 60% success rate in vulnerability exploitation and over 10% absolute improvement in vulnerability detection. Ablation and iteration studies further confirm the critical role of execution feedback, structured interaction, and memory for building robust and generalizable cybersecurity agents.

1. Introduction

Red teaming plays a foundational role in modern cybersecurity by proactively identifying, exploiting, and mitigating vulnerabilities (Dasgupta et al., 2022; Jang-Jaccard and Nepal, 2014; Sun et al., 2018) before they are abused in real-world attacks. Systematic red-teaming efforts help organizations assess their security posture, validate defenses, and reduce potential financial and operational losses caused by software vulnerabilities (Bhamare et al., 2020; Sarker et al., 2020). Standardized frameworks like the Common Weakness Enumeration (CWE) (MITRE Corporation, 2025) and the OWASP Top 10 (OWASP Foundation, 2021) systematize recurring software flaws, highlighting the widespread prevalence and severity of vulnerabilities in deployed systems. In practice, however, effective red teaming remains a complex and labor-intensive process that requires deep domain expertise, iterative hypothesis testing, and careful reasoning across large codebases and system configurations. Manual red-teaming workflows are time-consuming, costly, and difficult to scale, making it challenging to provide timely and comprehensive security assessments for rapidly evolving software systems Singhania et al. (2025). These limitations motivate the development of automated red-teaming techniques that can continuously and systematically uncover vulnerabilities at scale.

Recent advances in large language models (LLMs) (Achiam et al., 2023; Comanici et al., 2025) have sparked growing interest in automating aspects of cybersecurity analysis, including vulnerability discovery and exploitation (Xu et al., 2025; Zhang et al., 2025b). LLMs and agent systems offer a promising foundation for automatic red teaming due to their ability to reason over code, generate exploits, and interact with complex environments (Ferrag et al., 2025; He et al., 2024; Wei et al., 2022). However, existing approaches based on individual LLMs, single-agent setups (Guo et al., 2025; Zhang et al., 2024), or generic coding agents (Team, 2024) often fall short when applied to realistic security tasks. For example, empirical evidence from established benchmarks such as CyBench (Zhang et al., 2024), BountyBench (Zhang et al., 2025a), and CyberGym (Wang et al., 2025) shows that these systems struggle with multi-step reasoning, adaptive attack planning, and robust exploration of

Corresponding authors: Corresponding authors: Pengfei He hepengf1@msu.edu and Long T. Le: longtle@google.com
* This work was done while Pengfei was a Student Researcher at Google Cloud AI Research.

the vulnerability space. Addressing these challenges requires automated red-teaming systems that can decompose complex security workflows into specialized roles, support structured interaction, and iteratively critique and refine intermediate hypotheses. LLM-based multi-agent systems naturally meet these requirements, offering a more principled and effective framework for automated red teaming in software security.

Contribution. We propose Co-RedTeam, a security-aware multi-agent framework for automatic software vulnerability discovery and exploitation, explicitly designed to overcome core limitations of existing LLM-based security systems, namely brittle single-shot reasoning, lack of execution-grounded validation, and the inability to learn from prior attacks. Inspired by how human security experts conduct red teaming, Co-RedTeam tightly integrates four capabilities essential for realistic cybersecurity tasks: *security grounding*, *code-aware analysis*, *execution-driven reasoning*, and *experience accumulation*. Concretely, agents are grounded in established security standards and vulnerability documentation (e.g., CWE and OWASP) and equipped with code-browsing tools to precisely analyze large, complex codebases and generate evidence-backed vulnerability hypotheses. To move beyond static analysis, the framework employs a closed-loop planning–execution–evaluation process, enabling agents to iteratively refine exploitation strategies based on real execution feedback in isolated environments. Finally, Co-RedTeam introduces a layered long-term memory that captures reusable vulnerability patterns, high-level attack strategies, and concrete technical actions, allowing the system to improve over time. Together, these design choices directly address the shortcomings of prior automatical red-teaming approaches, yielding a principled, execution-grounded, and experience-driven framework tailored to real-world software security analysis.

To validate effectiveness, we evaluate Co-RedTeam on recent and challenging cybersecurity benchmarks, including CyBench (Zhang et al., 2024), BountyBench (Zhang et al., 2025a), and CyberGym (Wang et al., 2025). Experimental results demonstrate that Co-RedTeam substantially outperforms prior approaches, achieving over **60%** exploitation success and **20%** detection accuracy. We further conduct ablation studies to verify the importance of key design components and demonstrate that Co-RedTeam continually improves over time through long-term memory.

2. Related works

LLMs for Cybersecurity tasks. Software vulnerabilities, such as injection flaws, improper access control, and insecure deserialization, remain a fundamental challenge in cybersecurity, as formalized by widely adopted standards and benchmarks including CWE (MITRE Corporation, 2025), the OWASP Top 10 (OWASP Foundation, 2021), and OSS-Fuzz (Google, 2016). Recent advances in code-capable large language models (LLMs) (Achiam et al., 2023; Comanici et al., 2025) have spurred growing interest in using LLMs for vulnerability-related tasks, including detection, exploitation, and repair (Xu et al., 2025; Zhang et al., 2025b). Early studies show promising results, demonstrating that LLMs can identify certain vulnerability patterns and, in controlled settings, even autonomously exploit websites (Fang et al., 2024), with chain-of-thought prompting further improving performance in vulnerability discovery and repair (Nong et al., 2024). However, subsequent empirical evaluations reveal substantial limitations: LLMs struggle with complex reasoning in vulnerability detection (Ding et al., 2024), and both detection and exploitation remain challenging across diverse benchmarks and realistic settings (Steenhoek et al., 2024; Ullah et al., 2024; Zhou et al., 2024).

Agentic Systems for Security. Beyond single-LLM approaches, recent work increasingly adopts agentic system designs that structure LLMs as autonomous agents capable of interacting with tools, environments, and intermediate feedback (Huang et al., 2024; Liu et al., 2023). While training-based methods can be promising (Zhuo et al., 2025), agentic approaches are often favored for their

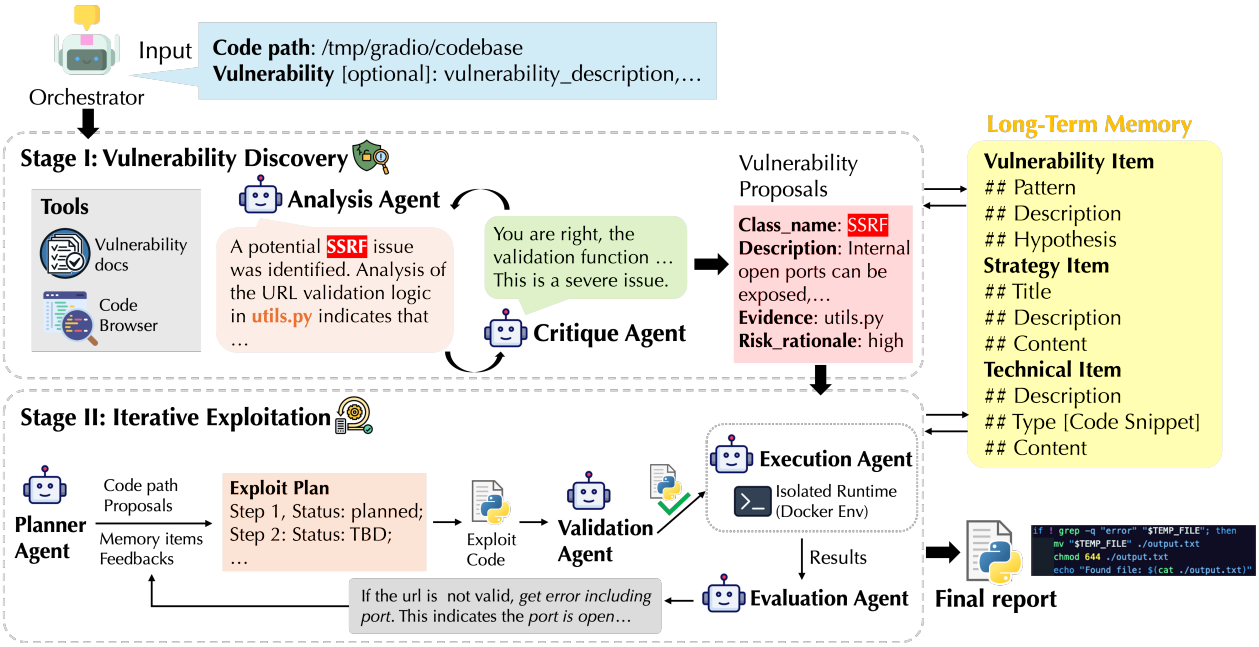

Figure 1 | Overview of Co-RedTeam. Co-RedTeam is a security-aware multi-agent framework for automatic vulnerability discovery and exploitation. **(Top)** Given a target codebase (and optional vulnerability hints), the orchestrator coordinates two stages. **Stage I (Vulnerability Discovery):** Analysis and Critique agents discuss together leveraging code-browsing tools and security documentation to identify and validate candidate vulnerabilities with concrete evidence. **Stage II (Iterative Exploitation):** Planning, Validation, Execution, and Evaluation agents interact with an isolated execution environment to iteratively reproduce vulnerabilities through execution-grounded feedback. Throughout the process, a layered *long-term memory* stores vulnerability patterns, high-level strategies, and concrete technical actions, enabling experience reuse and continual improvement across tasks.

flexibility, modularity, and ability to directly leverage rapidly advancing state-of-the-art LLMs without retraining. Prior efforts include single-agent security workflows that iteratively analyze codebases and refine vulnerability hypotheses (Ding et al., 2024; Guo et al., 2025; Zhang et al., 2024), but their effectiveness remains limited for complex, multi-step tasks. More recent work explores multi-agent designs, where agents collaborate via structured interaction or task decomposition, such as mock-court–style vulnerability detection (Widyasari et al., 2025) and coordinated exploitation of real-world one-day vulnerabilities (Fang et al., 2024). However, results on challenging benchmarks (Wang et al., 2025; Zhang et al., 2024, 2025a) show that existing single-agent and generic coding-agent systems achieve low success rates (often below 10%) on large, realistic codebases, motivating the need for security-aware multi-agent LLM systems.

3. Co-RedTeam

We propose multi-agent framework, Co-RedTeam, for automatic software vulnerability discovery and exploitation, designed to operate on real-world codebases and execution environments. As illustrated in Figure 1, the system is coordinated by an *orchestrator* that takes as input a target codebase (specified by a code path) and, optionally, a vulnerability description (Section 3.1). Co-RedTeamoperates in two sequential stages: (i) **vulnerability discovery**, where multiple agents collaboratively analyze code, retrieve vulnerability documents and learn from previous experience to generate structured vulnerability hypotheses (Section 3.2), and (ii) **iterative exploitation**, where

candidate vulnerabilities are validated through *execution-driven* planning, feedback, and refinement (Section 3.3). The system outputs a vulnerability report consisting of validated vulnerabilities. Throughout both stages, CO-REDTEAM maintains a shared long-term memory that accumulates experience across tasks, enabling continual improvement over time (Section 3.4). We present detailed designs in the following sections, and implementation details are shown in Appendix A.1.

Problem setup. We study the task of *software vulnerability analysis*, where an automated system is given access to a target *codebase* and an associated *execution environment*. The system is required to (i) **identify potential security vulnerabilities** grounded in concrete, code-level evidence when no explicit vulnerability description is provided, and (ii) **validate identified vulnerabilities** by reproducing their impact through execution. This setting emphasizes integrated capabilities in code analysis, security-domain reasoning, exploit planning, and execution-driven validation. Representative task examples are provided in Appendix A.5.

3.1. Orchestrator and System Initialization

Automated vulnerability discovery and exploitation require coordinating code inspection, security reasoning, and real execution feedback across multiple stages—challenges that previous methods often fail to handle reliably. At the core of CO-REDTEAM lies the *orchestrator*, which addresses these challenges by acting as a central controller that transforms high-level security objectives into a coordinated, execution-ready attack workflow (top part of Figure 1). Rather than treating agents as independent workers, the orchestrator enforces structure, discipline, and control, for reliable automated red teaming.

The orchestrator is responsible for initializing all agent instances and configuring their capabilities through *role-aware tool assignment*. Agents engaged in vulnerability discovery are equipped with code-browsing and vulnerability-documentation tools, along with critique utilities, enabling careful inspection and evidence-backed reasoning. In contrast, agents responsible for execution are granted access to sandboxed interfaces such as `run-bash` and `run-python` within an isolated Docker environment, ensuring that exploitation attempts are both realistic and contained. Agent instances and their tool handles are cached by the orchestrator, allowing consistent coordination and reuse across stages. Before any analysis begins, the orchestrator validates the user-provided inputs, including verifying the existence of the target codebase and inspecting optional vulnerability descriptions or hints. Based on this validation, it dynamically determines the appropriate execution path, either initiating the vulnerability discovery stage when no reliable hypothesis is available or directly proceeding to iterative exploitation when sufficient guidance is provided. Throughout the process, the orchestrator continuously monitors system state and schedules agent interactions, advancing the workflow as intermediate conditions are met. For example, once a vulnerability is successfully reproduced through execution, the orchestrator halts further exploration and transitions the system to final report generation. Similar in spirit to the supervisor agent used in Co-Scientist (Gottweis et al., 2025), this design enables the orchestrator to enforce role separation, control tool access, and manage overall control flow, thereby supporting stable and effective automatic vulnerability analysis.

3.2. Stage I: Vulnerability Discovery

Stage I systematically explores the code files and identifies potential vulnerabilities. As illustrated in Figure 1, this stage is driven by an *Analysis agent*, supported by a *Critique agent* and a suite of code analysis and security knowledge tools. This stage is activated when no explicit vulnerability description is available.

Analysis Agent: evidence-grounded vulnerability hypothesis generation. While large language

models exhibit strong reasoning and coding abilities, prior work shows that LLMs alone struggle to reliably identify software vulnerabilities, especially in complex codebases with non-trivial structure and data flow (Ullah et al., 2024; Zhang et al., 2024). Our key insight is that effective vulnerability discovery requires *grounding reasoning in both program structure and security-domain knowledge*. To this end, we equip the Analysis agent with code-browsing tools that enable systematic exploration of the codebase, including inspection of file hierarchies, documentation, entry points, configuration files, and fine-grained code snippets. These capabilities allow the agent to build a global understanding of program structure and data flow rather than relying on localized pattern matching. In parallel, the Analysis agent is connected to structured security knowledge sources distilled from widely adopted standards such as CWE and OWASP (details in Appendix A.2). This integration enables the agent to interpret suspicious code fragments through the lens of known vulnerability classes and exploitation mechanisms. Guided by this combined structural and security context, the agent performs a deep analysis of candidate code regions, explicitly reasoning about how untrusted inputs propagate through the program, where they reach sensitive sinks, and why existing validation or sanitization mechanisms may be insufficient.

For each candidate vulnerability, the Analysis agent constructs a rigorous *evidence chain* that explicitly identifies the input source, the vulnerable sink, and the execution context that enables exploitation. This evidence-centric design ensures that vulnerability hypotheses are grounded in concrete, inspectable code behavior rather than superficial correlations. The output of this step is a structured list of vulnerability drafts, each annotated with a standardized vulnerability class, a concise description, file- and line-level evidence, and a rationale describing potential impact.

Critique Agent: internal validation and refinement. To further reduce false positives and improve robustness, vulnerability drafts are reviewed by a Critique agent. Acting as an independent verifier, the Critique agent evaluates each proposal by examining its description, evidence, and risk rationale, optionally consulting code-browsing tools or vulnerability documentation to verify context. Each candidate is assigned a risk level (Critical to Informational) and a review status, *approved*, *rejected*, or *needs refinement*, together with concrete feedback explaining the decision. Vulnerabilities lacking convincing evidence or presenting only low-impact issues are rejected, while plausible but under-supported hypotheses are flagged for refinement. In response to critique feedback, the Analysis agent revisits the codebase to strengthen evidence or discards unsupported candidates. This iterative analysis–critique loop continues until a stable set of well-supported vulnerabilities is obtained.

The final output of Stage I is a curated set of validated vulnerability candidates, each supported by explicit evidence and an assessed risk level. By combining code-aware analysis, security-domain knowledge, and structured internal critique, Stage I produces high-confidence vulnerability hypotheses that form a reliable foundation for downstream, execution-driven exploitation.

3.3. Stage II: Iterative Exploitation

Vulnerability discovery can often be performed through static code reasoning, but *a precise validation fundamentally requires execution*. In practice, exploitation attempts frequently fail due to incomplete assumptions, missing context, or environment-specific constraints, making single-shot exploit generation unreliable. Stage II is therefore designed as an execution-grounded, iterative process that systematically refines exploitation strategies based on real execution feedback, transforming candidate vulnerabilities into concrete, reproducible evidence. As illustrated in Figure 1, Stage II operates as a tightly coupled, closed-loop process coordinated by the orchestrator and driven by three specialized agents: a *Planner* agent, an *Execution* agent, and an *Evaluation* agent. Rather than attempting single-shot exploit generation, the system treats exploitation as a structured search process guided by real execution feedback, continuing until a vulnerability is successfully reproduced or